MeetFlow

Phase 1 Implementation Plan

One-to-One P2P Video Baseline

Detailed Task Division & Startup Guide for a 2-Member Team

MeetFlow Engineering Team

June 23, 2026

Contents

1 Executive Summary **3**

2 Team Roles & Responsibility Matrix **3**
2.1 Engineer A: Backend & Infrastructure . . . 3
2.2 Engineer B: Frontend & WebRTC Client . . . 3
2.3 Collaborative Milestones . . . 4

3 Step-by-Step Startup Guide **5**
3.1 Step 1: Preamble & Agreement on the Signaling Protocol (Both Engineers) . . . 5
3.2 Signaling Protocol Walkthrough . . . 5
3.3 Step 2: Backend Core Setup (Engineer A) . . . 6
3.4 Step 3: Frontend Client Setup (Engineer B) . . . 6

4 Detailed Task Breakdown & Timeline **7**

5 Signaling Server Interface Contract **8**
5.1 Client Emit: Join Room . . . 8
5.2 Server Broadcast: User Joined . . . 8
5.3 Client Emit: SDP Negotiation (Offer / Answer) . . . 8
5.4 Client Emit: ICE Candidate . . . 8

6 Testing, Diagnostics & Debugging Tips **9**
6.1 Recommended WebRTC Debugging Tools . . . 9
6.2 Common Gotchas & Solutions . . . 9

1 Executive Summary

This document details the development roadmap and role assignment for executing **Phase 1 (One-to-One Baseline)** of the MeetFlow project.

The primary objectives of Phase 1 are:

- Establish a robust Client-Server Socket.IO signaling connection.
- Implement dynamic room creation and joining APIs.
- Set up the WebRTC `RTCPeerConnection` lifecycle to negotiate and stream peer-to-peer audio and video directly between two browsers.

This guide divides tasks for a **two-member engineering team** (referred to as **Engineer A: Backend & Infrastructure** and **Engineer B: Frontend & WebRTC Client**).

2 Team Roles & Responsibility Matrix

To complete Phase 1 efficiently, development is split into parallel paths, converging during integration and testing.

2.1 Engineer A: Backend & Infrastructure

Engineer A is responsible for the foundation server, room management logic, and the WebSocket signaling gateway.

- **Tech Stack Focus:** Node.js, Express, Socket.IO, CORS configuration.
- **Key Tasks:**
  1. Set up the Node.js development server with environment configurations.
  2. Create HTTP endpoints to generate and validate unique Room IDs.
  3. Implement the Socket.IO server to orchestrate signaling events (`join-room`, relaying `offer`, `answer`, and `ice-candidate`).
  4. Manage transient room states (e.g., tracking socket IDs of participants per room).

2.2 Engineer B: Frontend & WebRTC Client

Engineer B is responsible for the user interface, media device acquisition, socket connection client, and WebRTC peer connection management.

- **Tech Stack Focus:** React, Vite, Socket.IO-Client, WebRTC API.
- **Key Tasks:**
  1. Bootstrap the React + Vite application and set up basic routing/views.
  2. Build the UI forms to create a new meeting room or join an existing one.
  3. Implement local camera and microphone stream capture via `navigator.mediaDevices.getUserMedi`
  4. Orchestrate the `RTCPeerConnection` state machine (handling track additions, local/remote description configuration, and ICE candidates).
  5. Design the media stage grid displaying the local and remote HTML video elements.

2.3 Collaborative Milestones

Both engineers must pair-program on the following critical integration steps:

- **Signaling Contract Definition:** Agreeing on exact Socket.IO event names and JSON payload schemas.
- **End-to-End P2P Negotiation:** Debugging the SDP offer/answer handshakes and ICE candidate exchange.
- **Local Network Testing:** Conducting multi-device tests (e.g., phone to laptop, or two browser tabs) on the same local area network (LAN).

3 Step-by-Step Startup Guide

3.1 Step 1: Preamble & Agreement on the Signaling Protocol (Both Engineers)

Before writing any code, both engineers must finalize the signaling protocol contract. The standard exchange pattern is diagrammed below:

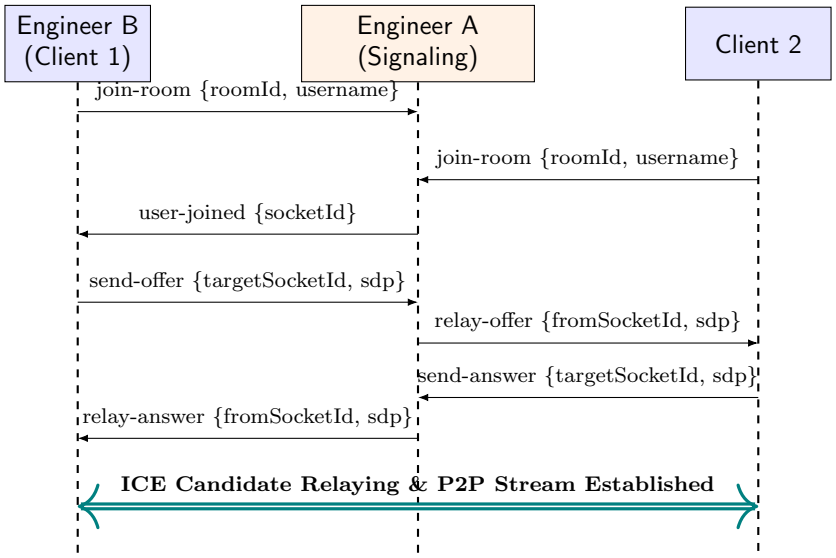

Figure 1: Phase 1 Signaling and Connection Protocol

3.2 Signaling Protocol Walkthrough

To ensure clear alignment between Engineer A and Engineer B, the sequence of events highlighted in Figure 1 must execute exactly as follows:

1. `join-room` **(Client 1 → Server):** When the first user (Client 1, developed by Engineer B) opens the application and enters a room, the client emits a socket event containing the `roomId` and their chosen `username`. The server registers Client 1's socket session and adds it to the corresponding room room pool.

2. `join-room` **(Client 2 → Server):** When the second user (Client 2) enters the same room, their client emits a matching `join-room` event. The server registers Client 2 and checks that the occupancy of the room does not exceed 2 (the limit for Phase 1's one-to-one baseline).

3. `user-joined` **(Server → Client 1):** Once Client 2 has joined successfully, the signaling server emits a `user-joined` event *exclusively* to the first client (Client 1). This packet carries the unique `socketId` of Client 2, informing Client 1 that a remote peer is present and ready to initiate the WebRTC handshake.

4. `send-offer` **(Client 1 → Server):** Upon receiving `user-joined`, Client 1 initializes their local `RTCPeerConnection`, requests camera/microphone access, attaches these local media tracks to the connection, and calls `createOffer()`. The resulting Session Description Protocol (SDP) offer is sent to the server in a `send-offer` packet, addressed to Client 2's socket ID.

5. `relay-offer` **(Server → Client 2):** The server acts as a relay, receiving the `send-offer` payload and instantly forwarding it as a `relay-offer` event to the target client (Client 2). This packet contains Client 1's original SDP offer and their source socket ID.

6. `send-answer` **(Client 2 → Server):** Client 2 receives the SDP offer, applies it as their Remote Description, initializes their own camera/microphone streams, adds their media tracks to their local `RTCPeerConnection`, and calls `createAnswer()`. The generated local SDP answer is wrapped in a `send-answer` packet and sent back to the server, targeting Client 1's socket ID.

7. `relay-answer` **(Server → Client 1):** The server receives the answer and relays it to Client 1 as a `relay-answer` event. Client 1 receives this answer and applies it as their Remote Description, completing the initial SDP negotiation.

8. **ICE Candidate Relaying & P2P Stream Established (Client 1 ↔ Client 2):** Concurrently, as each client discovers viable local network interfaces and router mappings (via public STUN servers), their respective browser WebRTC engines fire `onicecandidate` events. These candidates are relayed back and forth through the signaling server. Once a compatible connection pathway is successfully verified, the clients establish a secure, direct peer-to-peer connection (using DTLS/SRTP) to stream audio and video tracks without routing media through the backend server.

3.3 Step 2: Backend Core Setup (Engineer A)

Engineer A initiates the workspace backend folder structure:

```
mkdir server
cd server
npm init -y
npm install express socket.io cors dotenv
```

Configure an `index.js` that boots an HTTP server listening on port `5000` and binds Socket.IO with CORS settings allowing traffic from the frontend development port (typically `http://localhost:5173`).

3.4 Step 3: Frontend Client Setup (Engineer B)

Engineer B bootstraps the client folder layout in parallel:

```
npm create vite@latest client -- --template react
cd client
npm install socket.io-client react-router-dom lucide-react
```

Design the routing layout using React Router:

- `/` → Landing Screen (Join Room form / Create Room button).
- `/room/:roomId` → Video Conference Room.

4 Detailed Task Breakdown & Timeline

Day	Assignee	Task Description
Day 1	Eng A	Setup basic Express app, configure environmental variables, and build a REST API (`POST /api/rooms`) returning a unique 9-digit hyphenated Room ID (`XXX-XXX-XXX`).
	Eng B	Create frontend boilerplate with routing. Design the landing page containing a text input for Room ID, user nickname, and buttons.
Day 2	Eng A	Initialize Socket.IO connection handlers. Write room state management to prevent more than two users from joining the same room (One-to-One boundary).
	Eng B	Add camera and microphone selection logic. Render the local camera track onto a `<video>` component with audio muted locally to prevent feedback.
Day 3	Eng A	Complete signaling event listeners (`offer`, `answer`, `ice-candidate`) that target a specific peer socket.
	Eng B	Build the Socket manager in React using React contexts or custom hooks. Verify connection states, connect client sockets, and join the correct room.
Day 4	Both	**Integration Day 1**: Link frontend client sockets to the backend. Verify logging on server when a user enters/leaves, and check user count limits.
Day 5	Eng B	Write the WebRTC core handshake: Instantiate `RTCPeerConnection`, attach media tracks, capture generated local ICE candidates, and trigger SDP offer generation upon receiving `user-joined`.
	Eng A	Assist with real-time logging, testing websocket packet integrity, and configuring fallback STUN servers.
Day 6	Both	**Integration Day 2**: Verify the complete WebRTC handshake. Configure the remote stream component, bind incoming tracks to the `remoteVideo` HTML element, and test audio/video flow.
Day 7	Both	Perform local loopback tests, verify NAT traversal on different browser engines (Chrome vs Firefox/Safari), and package codebase for Phase 2.

Table 1: Phase 1 Action Schedule

5 Signaling Server Interface Contract

To prevent runtime configuration disparities, the Socket.IO communication must adhere to these exact structures:

5.1 Client Emit: Join Room

Emitted by the client immediately upon mounting the room page.

```
{
  "roomId": "abc-123-xyz",
  "username": "EngineerB"
}
```

5.2 Server Broadcast: User Joined

Broadcasted by the server to the existing socket in the room when a second peer joins.

```
{
  "socketId": "socket_id_of_new_peer",
  "username": "EngineerB"
}
```

5.3 Client Emit: SDP Negotiation (Offer / Answer)

Clients wrap descriptions in these formats before sending them to the signaling relay:

```
{
  "targetSocketId": "socket_id_of_recipient",
  "sdp": {
    "type": "offer", // or "answer"
    "sdp": "v=0\r\no=- 52362354256..."
  }
}
```

5.4 Client Emit: ICE Candidate

Clients must package network paths immediately upon discovery:

```
{
  "targetSocketId": "socket_id_of_recipient",
  "candidate": {
    "candidate": "candidate:842163049 1 udp 1686052607 192.168.1.100...
      ",
    "sdpMid": "0",
    "sdpMLineIndex": 0
  }
}
```

6 Testing, Diagnostics & Debugging Tips

To resolve WebRTC issues quickly during development, follow these standard diagnostic procedures:

6.1 Recommended WebRTC Debugging Tools

- **Chrome Internal Page:** Access `chrome://webrtc-internals/` in your browser tab during an active session. It provides real-time graphs showing bytes sent/received, connection states, and ICE selection processes.
- **Firefox WebRTC Tooling:** Navigate to `about:webrtc` for connection logs.

6.2 Common Gotchas & Solutions

1. **No Audio/Video Flowing:** Check if the browser block settings are active. WebRTC requires secure origins. Use `http://localhost` for local testing; if testing over a local network, configure the backend and frontend servers on HTTPS, or bypass browser policies in Chrome via: `chrome://flags/#unsafely-treat-insecure-origin-as-secure`.
2. **ICE Connection Fails:** Ensure you use public STUN servers in your client setup. A free, public server like `stun:stun.l.google.com:19302` must be supplied in your `RTCConfiguration` object:

```
const peerConnection = new RTCPeerConnection({
  iceServers: [
    { urls: 'stun:stun.l.google.com:19302' }
  ]
});
```

3. **Local Audio Feedback:** Ensure the local HTML `<video>` element has the `muted` attribute enabled. Do not mute the remote video stream.